

24 — Embedding Cache:

Database-Backed Deduplication for API Calls

If you haven't read docs 06 and 11 yet, doc 06 covers what embeddings are and the provider pattern, and doc 11 covers batching and retry — this doc assumes that background and focuses on a different optimization: how do we avoid paying for (and waiting for) the same embedding twice?

What it is

`bilingual_etl/load/embeddings.py` includes a database-backed cache that stores every embedding it computes alongside a SHA-256 hash of the input text. Before calling the embedding API for a text, it checks the cache: "Have I already embedded this exact string?" If yes, it returns the stored vector instantly from the database. If no, it calls the API, stores the result in the cache, and then returns it.

This is not a generic caching library — it's a single-purpose, content-addressed lookup table in PostgreSQL. The "address" is the SHA-256 hash of the text, and the "content" is the embedding vector.

Real-life analogy

Imagine you run a translation agency. A client sends you a stack of 1,000 documents to translate. You charge per document. But you notice that 300 of those documents are exact duplicates — the same memo sent to different departments.

Without a cache, you translate all 1,000, charging the client (or in our case, burning API quota) for 300 translations you've already done before.

With a cache, you keep a filing cabinet of every translation you've ever completed, indexed by the original document's fingerprint (a hash). Before translating a new document, you check the cabinet: "Have I translated this exact document before?" If the fingerprint matches, you pull out the existing translation — no work needed, no cost incurred. You only translate the 700 genuinely new documents.

The filing cabinet is the `embedding_cache` table. The fingerprint is the SHA-256 hash. The translation is the embedding vector.

Why we used it here

Three real scenarios in this project produce duplicate embedding requests:

1. **Re-running the ETL with `--force-enrichment`** . Force mode reprocesses every record's translation and enrichment, which may produce the same narrative text as the previous run (the source data hasn't changed, so the narrative is identical). Without a cache, every forced re-run would call the embedding API for thousands of texts it's already embedded. With the cache, those calls become instant database lookups.
2. **Overlapping text across records.** Some category names appear in multiple company narratives. If the narrative builder produces an identical text chunk for two different companies (same category description block), the second one is a cache hit.
3. **Partial failures and restarts.** If the ETL crashes halfway through (network failure, rate-limit exhaustion), you restart it. Records that were already processed had their embeddings computed and cached. The hash gate (`has_changed`) skips records whose source data hasn't changed, but if you're using `--force-enrichment` , the hash gate is bypassed — the embedding cache is the second line of defense, preventing redundant API calls for records where

the output narrative hasn't changed even though force mode is reprocessing everything.

The cost savings are real: Gemini embedding calls are free-tier but rate-limited (requests per minute), and Jina gives 10M free tokens. Every cache hit is one fewer API call counting against those limits, and one fewer round-trip of network latency.

Code walkthrough

From `bilingual_etl/load/embeddings.py`:

Hashing the input text:

```
def _text_hash(text: str) -> str:
    return hashlib.sha256(text.encode("utf-8")).hexdigest()
```

SHA-256 produces a 64-character hexadecimal string that is, for all practical purposes, unique to the input text. Even a single character change in the input produces a completely different hash. This is how we "fingerprint" each text without storing the full text as a lookup key (which would be expensive to compare and index for long narratives).

The cache table:

```
def _ensure_cache_table(conn) -> None:
    with conn.cursor() as cur:
        cur.execute("""
            CREATE TABLE IF NOT EXISTS embedding_cache (
                text_hash TEXT PRIMARY KEY,
                embedding VECTOR,
                created_at TIMESTAMPTZ DEFAULT NOW()
            )
        """)
    conn.commit()
```

Three columns: the hash as primary key (fast lookups, guaranteed unique), the embedding vector stored as pgvector's native `VECTOR` type (same as in `category_vectors` and `company_vectors`), and a

timestamp so you can see when each embedding was first computed.

`CREATE TABLE IF NOT EXISTS` means this runs safely on every ETL invocation — it creates the table the first time and does nothing on subsequent runs.

Cache lookup:

```
def _lookup_cached(conn, hashes: list[str]) -> dict[str, list[float]]:
    if not hashes:
        return {}
    placeholders = ",".join(["%s"] * len(hashes))
    with conn.cursor() as cur:
        cur.execute(
            f"SELECT text_hash, embedding::text FROM embedding_cache "
            f"WHERE text_hash IN ({placeholders})",
            tuple(hashes),
        )
    result = {}
    for row in cur.fetchall():
        vec_str = row[1].strip("[ ]")
        result[row[0]] = [float(x) for x in vec_str.split(",")]
    return result
```

This does a batch lookup — given a list of hashes, it fetches all matching cache entries in a single SQL query using `IN (...)`. The result is a dictionary mapping hash to embedding vector. Hashes not in the cache are simply absent from the dictionary. The `embedding::text` cast converts pgvector's binary format to a parseable string of comma-separated floats.

Cache storage:

```
def _store_cached(conn, items: list[tuple[str, list[float]]]) -> None:
    if not items:
        return
    with conn.cursor() as cur:
        for text_hash, embedding in items:
            vec_literal = "[" + ",".join(str(v) for v in embedding) + "]"
            cur.execute(
                "INSERT INTO embedding_cache (text_hash, embedding) "
                "VALUES (%s, %s::vector) ON CONFLICT (text_hash) DO NOTHING",
```

```

        (text_hash, vec_literal),
    )
    conn.commit()

```

`ON CONFLICT (text_hash) DO NOTHING` is the key detail: if two threads or two ETL runs try to cache the same text simultaneously, the second insert silently does nothing instead of raising a duplicate-key error. This makes the cache safe for concurrent use without locks.

The main flow in `embed_texts` :

```

def embed_texts(texts, provider=None, batch_size=DEFAULT_BATCH_SIZE, cache_conn=None):
    if not texts:
        return []

    provider = provider or get_embedding_provider()

    if cache_conn:
        try:
            _ensure_cache_table(cache_conn)
        except Exception:
            cache_conn = None            # Cache creation failed? Continue without

    hashes = [_text_hash(t) for t in texts]
    cached = _lookup_cached(cache_conn, hashes) if cache_conn else {}
    cache_hits = sum(1 for h in hashes if h in cached)

    uncached_indices = [i for i, h in enumerate(hashes) if h not in cached]
    uncached_texts = [texts[i] for i in uncached_indices]

    if uncached_texts:
        new_embeddings = []
        for i in range(0, len(uncached_texts), batch_size):
            batch = uncached_texts[i : i + batch_size]
            new_embeddings.extend(_embed_batch_with_retry(batch, provider))

        to_cache = []
        for idx, emb in zip(uncached_indices, new_embeddings):
            cached[hashes[idx]] = emb
            to_cache.append((hashes[idx], emb))

        if cache_conn and to_cache:
            try:

```

```
        _store_cached(cache_conn, to_cache)
    except Exception as e:
        logger.warning(f"Failed to write embedding cache: {e}")

    results = [cached[h] for h in hashes]
    # ... logging ...
    return results
```

Walking through it step by step:

1. Hash every input text.
2. Look up all hashes in the cache table (one SQL query).
3. Split the input into two groups: texts already in the cache (hits) and texts not in the cache (misses).
4. Only call the embedding API for the misses — using the batching and retry logic from doc 11.
5. Store the newly computed embeddings in the cache for next time.
6. Reassemble the results in the original order (both cached and freshly computed) and return them.

Cache failure is non-fatal. Notice the `try/except` blocks around both the table creation and the cache write. If the cache is unavailable (connection issue, permissions problem), the code falls back to computing all embeddings from the API — slower, but correct. The cache is a performance optimization, not a correctness requirement.

Cache invalidation strategy

This cache uses a simple but effective invalidation approach: **content-addressed storage means stale entries are never returned**. If the text changes by even one character, its SHA-256 hash changes, so the lookup will miss and the new text gets a fresh embedding from the API. Old entries for the previous version of the text remain in the table but are never looked up again — they're harmlessly dead.

This means the cache grows over time but never serves stale data. For this project's scale (thousands of records, not millions), the growth is negligible. If it ever became a concern, you could add a cleanup job

that deletes entries older than a certain `created_at` threshold, but that's not needed at current scale.

How to explain this in an interview

"We built a database-backed embedding cache to avoid redundant API calls. Before calling the embedding provider, we SHA-256 hash each input text and check a PostgreSQL table for a matching entry. Cache hits skip the API entirely and return the stored vector. Cache misses go through the normal batching and retry path, and the results are stored for future lookups. The content-addressed design means we never serve stale embeddings — if the text changes, the hash changes, so it's automatically a cache miss. The cache is especially valuable during ETL re-runs and failure recovery: records that were already embedded don't burn API quota or rate-limit headroom. Cache failures are non-fatal — if the cache table is unavailable, the code falls back to calling the API for everything, so correctness is never compromised for performance."